Caption AI: AI-Powered Social Media Caption Generator

Project Description:

The Credit Card Approval Prediction System is a machine learning-based web application developed to predict whether a credit card application is likely to be approved or rejected based on the applicant's personal and financial information. The system helps automate the credit approval process by analysing multiple factors such as age, annual income, employment status, education, family status, housing type, and other relevant details.

The application is built using the Flask framework for the backend, while HTML and CSS are used to design an interactive and user-friendly interface. A trained Machine Learning model processes the user inputs and generates a prediction indicating whether the applicant is likely to receive credit card approval. The application also validates user input to ensure accurate predictions and provides the result instantly.

The primary objective of this project is to demonstrate how machine learning can assist financial institutions in making faster and more consistent credit approval decisions while reducing manual effort. The system provides a simple, efficient, and reliable solution for predicting credit card approvals based on historical data.

Scenarios:

Scenario 1: Credit Card Approval

A customer enters all the required personal and financial details into the application. The machine learning model analyses the information and predicts that the customer is eligible for credit card approval.

Scenario 2: Credit Card Rejection

A customer with low annual income and an unstable employment history submits an application. Based on the provided information, the system predicts that the application is likely to be rejected.

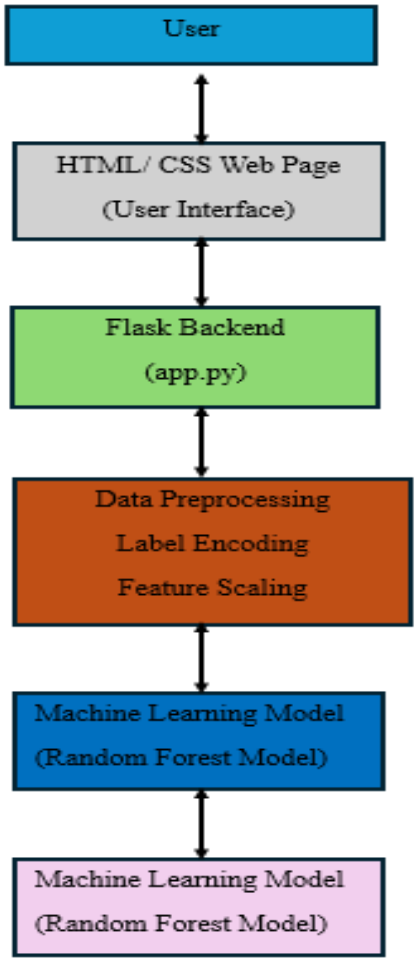
Scenario 3: Quick Decision Support

A bank employee uses the application to quickly evaluate multiple customer applications. The system instantly predicts the approval status, helping reduce manual effort and speed up the decision-making process.

Technical Architecture:

The Credit Card Approval Prediction System follows a three-tier architecture consisting of the presentation layer, application layer, and machine learning layer. The frontend is developed using HTML and CSS, providing a user-friendly interface for entering applicant details. The backend is built using the Flask framework, which receives the user input, performs data preprocessing, and communicates with the trained machine learning model. The machine learning model analyses the input data and predicts whether the applicant is likely to be approved or rejected for a credit card. The prediction result is then displayed to the user through the web interface.

Architecture Diagram:



Pre-requisites:

- Python Programming
- Flask Framework
- HTML and CSS
- Machine Learning Basics
- Pandas Library
- NumPy Library
- Scikit-learn Library
- Pickle Module
- Visual Studio Code
- Git and GitHub

Project Workflow:

Milestone 1: Data Collection and Preprocessing

- Activity 1.1: Collect the Credit Card Approval dataset from a reliable source.
- Activity 1.2: Explore the dataset and understand all input features and the target variable.
- Activity 1.3: Clean the dataset by handling missing values, duplicate records, and invalid data.
- Activity 1.4: Perform data preprocessing, including label encoding, feature scaling, and splitting the dataset into training and testing sets.

Milestone 2: Machine Learning Model Development

- Activity 2.1: Train multiple machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine (SVM).
- Activity 2.2: Compare the performance of all trained models using evaluation metrics.
- Activity 2.3: Select the best-performing model based on prediction accuracy.
- Activity 2.4: Save the trained machine learning model, scaler, and label encoders using Pickle files.

Milestone 3: Flask Application Development

- Activity 3.1: Develop the Flask backend using Python.
- Activity 3.2: Create application routes to display the home page and process prediction requests.
- Activity 3.3: Load the trained machine learning model, scaler, and label encoder files.
- Activity 3.4: Implement prediction logic to process user inputs and display the prediction result.

Milestone 4: Frontend Development

- Activity 4.1: Design the user interface using HTML and CSS.
- Activity 4.2: Create an applicant information form for entering personal and financial details.
- Activity 4.3: Display the prediction result in a simple and user-friendly format.
- Activity 4.4: Improve the interface by enhancing layout, readability, and responsiveness.

Milestone 5: Testing and Deployment

- Activity 5.1: Perform functional testing to verify that all application features work correctly.
- Activity 5.2: Conduct performance testing to evaluate the application's response time and stability.
- Activity 5.3: Run the Flask application locally and verify prediction results.
- Activity 5.4: Upload the complete project to GitHub and prepare the project documentation.

Milestone 6: Conclusion

- Activity 6.1: Review the overall project implementation and testing results.
- Activity 6.2: Confirm that the application successfully predicts credit card approval based on user inputs.

- Activity 6.3: Identify possible future improvements, such as deploying the application to a cloud platform, enhancing prediction accuracy with larger datasets, and adding user authentication.

Milestone 1: Data Collection and Preprocessing

In this milestone, the primary focus is on collecting the credit card approval dataset and preparing it for machine learning model development. The dataset is analysed to understand its features, cleaned to remove inconsistencies, and preprocessed to ensure that it is suitable for training accurate prediction models. Proper preprocessing improves the quality of the data and enhances the performance of the machine learning algorithms.

Activity 1.1: Collect the Credit Card Approval Dataset

The first step in the project is to collect a suitable dataset for predicting credit card approval. The dataset contains applicant information such as gender, age, income, employment status, family status, education level, housing type, and other financial details required for prediction. This dataset serves as the foundation for training and evaluating the machine learning models.

Steps:

1. Download the Credit Card Approval dataset from a reliable source.
2. Store the dataset in the project directory.
3. Load the dataset using the Pandas library.
4. Verify that all records and columns are loaded correctly.

Activity 1.2: Explore and Understand the Dataset

After loading the dataset, it is important to understand the structure and characteristics of the data. This helps identify important features, the target variable, missing values, and the overall quality of the dataset.

Steps:

1. Display the first few rows of the dataset.
2. Check the number of rows and columns.
3. Identify numerical and categorical features.
4. Analyse the target variable (Approved / Rejected).

Activity 1.3: Data Cleaning

Data cleaning is performed to improve data quality before training the machine learning model. Missing values, duplicate records, and invalid entries are identified and handled appropriately.

Steps:

1. Check for missing values.
2. Remove duplicate records.
3. Handle inconsistent or invalid data.
4. Ensure all required features are available for model training.

Activity 1.4: **Data Preprocessing**

Before training the model, the dataset is transformed into a suitable format for machine learning algorithms. Categorical variables are converted into numerical values, numerical features are scaled, and the dataset is divided into training and testing datasets.

Steps:

1. Apply Label Encoding to categorical features.
2. Perform Feature Scaling using StandardScaler.
3. Select input features and target variable.
4. Split the dataset into training and testing sets.

Milestone 2: Machine Learning Model Development

In this milestone, machine learning models are developed to predict whether a credit card application is likely to be approved or rejected. The preprocessed dataset is used to train multiple classification algorithms, and their performance is evaluated using appropriate metrics. The best-performing model is selected and saved for integration with the Flask web application.

Activity 2.1: Train Machine Learning Models

The cleaned and pre-processed dataset is used to train multiple machine learning algorithms. Different models are trained to compare their prediction performance and identify the most suitable model for the application.

Steps:

1. Import the required machine learning libraries.
2. Split the dataset into training and testing sets.
3. Train multiple classification models such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine (SVM).
4. Store the trained models for evaluation.

Activity 2.2: Evaluate Model Performance

After training, each model is evaluated to measure its prediction accuracy and overall performance. The evaluation helps identify the model that provides the most reliable results.

Steps:

1. Predict the output using the testing dataset.
2. Calculate the accuracy score for each model.
3. Compare the performance of all trained models.
4. Select the model with the highest accuracy.

Activity 2.3: Select the Best Model

1. Save the trained machine learning model.
2. Save the Standard Scaler object.
3. Save the Label Encoder objects.
4. Verify that all files are stored successfully for future use.

Activity 2.4: Save the Trained Model

The selected machine learning model and the required preprocessing objects are saved using Pickle files. These files are later loaded by the Flask application to generate predictions without retraining the model.

Steps:

1. Save the trained machine learning model.
2. Save the StandardScaler object.
3. Save the Label Encoder objects.
4. Verify that all files are stored successfully for future use.

Milestone 3: Flask Application Development

In this milestone, the backend of the Credit Card Approval Prediction System is developed using the Flask framework. The Flask application receives user input, processes the data, loads the trained machine learning model, and generates the prediction result. It acts as a bridge between the user interface and the machine learning model.

Activity 3.1: Set Up the Flask Application

The Flask framework is configured to create the backend of the web application.

Steps:

1. Install the Flask library.
2. Create the app.py file.
3. Initialize the Flask application.
4. Configure the project structure.

Activity 3.2: Create Application Routes

Routes are created to display the home page and process prediction requests.

Steps:

1. Create the home page route (/).
2. Create the prediction route (/predict).
3. Receive user input through the HTML form.
4. Return the prediction result to the user.

Activity 3.3: Load the Machine Learning Model

The trained model and preprocessing files are loaded into the Flask application.

Steps:

1. Load the trained model (credit_card_model.pkl).
2. Load the StandardScaler.
3. Load the Label Encoders.

4. Verify successful loading.

Activity 3.4: Generate Prediction

The application processes user input and generates the prediction.

Steps:

1. Read user input.
2. Apply preprocessing.
3. Predict using the trained model.
4. Display the result.

Milestone 4: Frontend Development

This milestone focuses on developing an interactive and user-friendly interface using HTML and CSS. The interface allows users to enter applicant details and view prediction results in a clear and organized manner.

Activity 4.1: Design the User Interface

A responsive user interface is created.

Steps:

1. Create the HTML page.
2. Design the application layout.
3. Add headings and labels.
4. Improve readability.

Activity 4.2: Create the Applicant Information Form

The form collects all required applicant information.

Steps:

1. Add input fields.
2. Add dropdown menus.
3. Add the Predict button.

4. Validate user inputs.

Activity 4.3: Display Prediction Result

The application displays the prediction after processing the data.

Steps:

1. Receive the prediction.
2. Show Approved/Rejected status.
3. Display applicant details.
4. Show prediction summary.

Activity 4.4: Improve User Experience

The interface is enhanced for better usability.

Steps:

1. Improve page layout.
2. Apply CSS styling.
3. Increase responsiveness.
4. Ensure easy navigation.

Milestone 5: Testing and Deployment

This milestone ensures that the application functions correctly under different conditions. Functional testing, performance testing, and deployment preparation are completed to verify system reliability.

Activity 5.1: Functional Testing

Each feature of the application is tested.

Steps:

1. Test all input fields.
2. Verify prediction accuracy.
3. Validate error handling.

4. Confirm application stability.

Activity 5.2: Performance Testing

The application performance is evaluated.

Steps:

1. Measure response time.
2. Test multiple requests.
3. Observe CPU and memory usage.
4. Record testing results.

Activity 5.3: Local Deployment

The application is executed locally.

Steps:

1. Run the Flask application.
2. Open the application in a browser.
3. Verify all functionalities.
4. Confirm successful execution.

Activity 5.4: Project Submission

The completed project is prepared for submission.

Steps:

1. Organize project files.
2. Upload the project to GitHub.
3. Prepare documentation.
4. Record the demo video.

Milestone 6: Conclusion

The final milestone summarizes the successful completion of the Credit Card Approval Prediction System. It highlights the project achievements, testing outcomes, and possible future enhancements.

Activity 6.1: Project Review

The overall project implementation is reviewed.

Steps:

1. Verify project objectives.
2. Review implementation.
3. Check documentation.
4. Confirm completion.

Activity 6.2: Result Verification

The application's prediction capability is verified.

Steps:

1. Test sample inputs.
2. Verify prediction results.
3. Confirm model performance.
4. Ensure application reliability.

Activity 6.3: Future Enhancements

Possible improvements are identified.

Steps:

1. Deploy the application on a cloud platform.
2. Improve model accuracy using larger datasets.
3. Add user authentication and login functionality.
4. Develop a mobile-friendly version of the application.